

Centro Regional Universitario Córdoba IUA

Maestría en Ciberdefensa



Trabajo Práctico N°3

Clave generada a partir de la fecha y hora

Módulo:

- Criptografía

Alumno:

- Ing. Vietto Herrera Santiago

Docentes:

- Ing. Montes Miguel

15 de Mayo de 2026

Córdoba - Argentina

Indice

Desarrollo	1
1. Problemas con la generación de claves	1
1.1. Introducción	1
1.2. El problema con la hora	1
2. Solución del desafío	2
2.1. Análisis del problema	2
2.2. Implementación de la solución	4
2.2.1. Conexión con servidor, solicitud del desafío y extracción de fecha y Base64	4
2.2.2. Conversión de la fecha del encabezado a timestamp Unix	6
2.2.3. Decodificación del cuerpo Base64 y separación de sus partes	6
2.2.4. Fuerza bruta sobre los microsegundos posibles, generación de claves y descifrado	7
2.2.5. Envío de la respuesta al servidor	10

Desarrollo

1. Problemas con la generación de claves

1.1. Introducción

Dentro de los diferentes casos de problemas derivados de mala generación de valores aleatorios, se menciona Netscape el cual era un browser popular en la época de los 90, antes de Internet Explorer. En el año 1996 generaba claves para el cifrado con el siguiente pseudo código:

```
RNG_CreateContext()
    (seconds, microseconds) = time of day; /* Time elapsed since 1970 */
    pid = process ID;  ppid = parent process ID;
    a = mklcpr(microseconds);
    b = mklcpr(pid + seconds + (ppid << 12));
    seed = MD5(a, b);

mklcpr(x) /* not cryptographically significant; shown for completeness */
    return ((0xDEECE66D * x + 0x2BBB62DC) >> 1);

MD5() /* a very good standard mixing function, source omitted */

RNG_GenerateRandomBytes()
    x = MD5(seed);
    seed = seed + 1;
    return x;
```

Básicamente tomaba la hora del día, en segundos y microsegundo, además tomaba el número de proceso que estaba ejecutando y el número del proceso padre. Todos estos datos se pasaban por un procesamiento, y generaba una semilla aplicando el algoritmo MD5 sobre esos valores.

Después, para generar valores sucesivos, lo que hacía era calcular el MD5 de la semilla e incrementar la semilla en 1. Es decir, tenía un contador en el que la semilla se incrementaba en 1 y se calculaba MD5.

1.2. El problema con la hora

El problema detrás de todo esto es que la hora no es aleatoria. En particular, si vemos un mensaje y sabemos a qué hora fue emitido el mensaje, tenemos un conjunto de información que nos alcanza para tratar de deducir cuál fue la clave que ha sido utilizada.

- El valor de seconds se puede obtener de la hora en la que fue generado el mensaje.
- El valor de microseconds puede tener 1 millón de valores posibles, es decir, aproximadamente 10^6 o 2^{20} valores.
- Si el atacante tiene acceso a la computadora donde se ejecuta Netscape, pid y ppid, son fáciles de obtener.
- Pero si el atacante no tiene acceso, por la forma en que se utilizan, solamente agregan 27 bits de entropía. Es decir, el atacante necesita realizar 2^{20} o a lo sumo 2^{47} operaciones para averiguar la clave (nada en términos computacionales), en lugar de necesitar hacer 2^{128} operaciones.

Como consecuencia de esto, utilizar valores predecibles como la hora, para inicializar un PRNG es una mala idea desde el punto de vista criptográfico y no debe ser utilizada bajo ninguna circunstancia, ya que no agrega nada. La hora nunca es impredecible, ya que es un contador que va creciendo en segundos.

2. Solución del desafío

2.1. Análisis del problema

El presente trabajo práctico tiene como objetivo explotar una debilidad en la generación de una clave criptográfica a partir de la fecha y hora de creación de un mensaje. En este caso, el servidor de la IUA devuelve, mediante una consulta GET a una URL asociada al correo electrónico registrado del alumno, un mensaje con formato similar a un correo electrónico. Este mensaje contiene un encabezado con la fecha de creación y un cuerpo codificado en Base64. Por ende, lo que se busca es descifrar el mensaje, recuperar el texto claro y enviarlo luego mediante un POST al servidor para verificar si la solución es correcta.

El problema existe porque la clave simétrica utilizada para cifrar el mensaje no fue generada con una fuente de aleatoriedad criptográficamente segura. Aunque el mensaje fue cifrado con AES-128 en modo CBC, la clave AES fue generada aplicando MD5 a la fecha y hora exacta de creación del mensaje expresada como tiempo Unix en microsegundos es decir, la cantidad de microsegundos segundos transcurridos desde las cero horas del 1 de enero de 1970 UTC). Es decir, la clave depende directamente de la hora, y como vimos anteriormente, este es un dato predecible.

El cuerpo del mensaje recibido está codificado en Base64. Al decodificarlo, no se obtiene directamente el mensaje claro, sino una estructura de bytes compuesta por tres partes:

128 bytes	16 bytes	n bytes
Clave simétrica cifrada	IV	Texto cifrado

- Los primeros 128 bytes corresponden a la clave simétrica cifrada con RSA de 1024 bits. En un escenario normal, esta parte sería necesaria para recuperar la clave AES mediante la clave privada RSA. Sin embargo, en este caso no se ataca RSA, porque la actividad nos dice que la clave AES fue generada a partir del timestamp del mensaje. Por lo tanto, la clave puede reconstruirse mediante fuerza bruta sobre los microsegundos posibles.
- Los siguientes 16 bytes corresponden al IV utilizado por AES-CBC. El IV no es la clave, pero es necesario para poder descifrar correctamente el primer bloque del mensaje. Como el IV está incluido en el cuerpo del desafío, no debe ser adivinado ni reconstruido, sino que se extrae y se reutiliza durante cada intento de descifrado.
- El resto corresponde al texto cifrado mediante AES-128-CBC. Esta es la parte que se intenta descifrar repetidamente con cada clave candidata generada a partir de los posibles microsegundos del timestamp.

El servidor proporciona en el encabezado una fecha como la siguiente:

- Date: Sat Jan 15 16:32:28 UTC 2022

Esa fecha tiene precisión de segundos. Sin embargo, la clave fue generada usando microsegundos. Por lo tanto, conocemos el segundo exacto en el que se creó el mensaje, pero desconocemos cuál fue el microsegundo exacto dentro de ese segundo. Esto deja solamente 1 millón de posibilidades posibles, desde el microsegundo 000000 hasta el 999999:

```
Sat Jan 15 16:32:28.000000 UTC 2022
Sat Jan 15 16:32:28.000001 UTC 2022
Sat Jan 15 16:32:28.000002 UTC 2022
...
Sat Jan 15 16:32:28.999999 UTC 2022
```

Por eso, aunque AES-128 normalmente debería ofrecer una seguridad equivalente a 128 bits, en este caso la seguridad real queda reducida a la cantidad de microsegundos posibles dentro de un segundo. Esto equivale aproximadamente a 20 bits de entropía, ya que:

- $\log_2(1000000) \approx 20$

Como sabemos, esto es muy poco para una computadora, por lo que se puede resolver mediante fuerza bruta.

Para cada microsegundo posible dentro del segundo indicado por la fecha del mensaje, se genera una clave candidata. Para ello, se toma el timestamp Unix en microsegundos, tomando el ejemplo serían 1642264348 segundos Unix. Entonces, desde el inicio de ese segundo en microsegundos, las claves candidatas se generan con:

```
1642264348000000
1642264348000001
1642264348000002
...
1642264348999999
```

Cada uno de estos valores se convierte a 8 bytes usando la convención big endian (algo como 00 00 00 00 00 00 00 00). Y luego se aplica MD5. Como MD5 produce una salida de 128 bits, esa salida se utiliza directamente como clave AES-128.

- $\text{MD5}(\text{timestamp_en_8_bytes}) = 16 \text{ bytes}$

Luego, con cada clave candidata, se intenta descifrar el texto cifrado usando AES-128 en modo CBC y el IV extraído del mensaje. Si la clave no es correcta, el resultado será basura, por ejemplo bytes sin sentido, pero si la clave es correcta, el resultado será el mensaje original más el padding PKCS7. Como AES cifra en bloques de 16 bytes, si el mensaje original no tiene una longitud múltiplo de 16, se agrega relleno al final. Por ejemplo, el siguiente mensaje tiene 13 bytes, y PKCS7 agrega 3 bytes con valor 03, si faltarían 5 bytes agregaría cinco 05, si fueran 10 agregaría diez 0A, y así sucesivamente:

- HOLA SANTIAGO 03 03 03

Entonces, como el cifrado utiliza relleno PKCS7, después de descifrar se intenta remover ese padding. Si el padding es inválido, la clave candidata se descarta porque es incorrecta. Si el padding es válido y además el resultado es texto ASCII legible, entonces se encontró la clave correcta y, por lo tanto, el mensaje claro.

2.2. Implementación de la solución

2.2.1. Conexión con servidor, solicitud del desafío y extracción de fecha y Base64

Dado a lo entendido en la sección 2.1, como primer paso tenemos la solicitud del desafío al servidor:

1. Se definen las credenciales de acceso al servidor, que son por un lado el correo electrónico registrado del alumno que lleva a cabo el siguiente trabajo práctico, y la URL del servidor.

```
12
13 server = "https://cripto.iua.edu.ar"
14 email = "santiagovietto5@gmail.com"
15
```

2. Luego, se declara una función `get_challenge()` para hacer posible la comunicación con el servidor y solicitar a este el desafío mediante un GET, .

```
19
20 def get_challenge():
21     response = requests.get(f"{server}/timerand/{email}/challenge")
22     response.raise_for_status()
23     return response.text
24
```

3. Hacemos un llamado a la función `get_challenge()` almacenando el contenido en una variable `challenge`. Y mostramos el contenido obtenido por pantalla.

```
83
84 challenge = get_challenge()
85
86 date, body_base64 = separate_challenge(challenge)
87
88 print("\nRespuesta servidor:")
89 print(challenge)
90
```

```
(venv_TPScripto) santiago-vietto@santiago-vietto-pc:~/Documents/Trabajos-Practicos-Criptografia$ python TP3_Santiago_Vietto.py
Respuesta servidor:
From: User <user@example.com>
Date: Sat Jan 15 16:32:28 UTC 2022
To: santiagovietto5@gmail.com

MB4CXjEYyIZbuPP9jhXyRKYwvsVefL37KPxrS/dD6vrQRnqny2pURHcT8bKIterdZ9CLDeSgF/rRXTc32zgbZ68ATxYpC1AcH9w7Jng4AJf/yy+nDBKtgVksZZvHybQ
xziliZEGjn77IcHkrCR7mrBMcUk09q+1nm/TunyQ7ajfPuPIDMioICFTE0EiktTPrcZ6B46f1hhnFVSTpI6EJI9yHT/EoZcZuvkZ5xzTY+qd1mD69/1BksT8JdX+Qal
grgOLjkvRb+bWbj0oXBM1rKTHSpngCwiP0TnTLbtOKDFKpgICu86V6S2S6Wd1XdSu2EB0ni2h6R85cbxNU+JqHrTHIQ3i2kfDIXv8FA5dk8Cv2y/HcNHVQaTqvFc5a
31hWbz6KaToYL4AH8/SquQ==
```

3. Finalmente, extraemos la fecha del encabezado Date del mensaje obtenido y el cuerpo codificado en Base64. Para esto se pasa por parámetro el contenido de challenge a una función `separate_challenge()`, en la cual primero se convierte la respuesta completa en una lista de líneas, porque la fecha está en una línea concreta, y el cuerpo Base64 empieza después de una línea vacía. Y se inicializan 3 variables:

- `date`: de tipo `None` porque significa que todavía no encontró la fecha.
- `body_base64`: Una lista donde se va a ir guardando las líneas del cuerpo Base64.
- `reading_body`: Bandera en `False`, que indica que todavía está leyendo el encabezado, y no el cuerpo.

3.1. En el ciclo `for` se recorre línea por línea:

- Primero se busca la línea que empieza con "Date:". Y si la encuentra, le quita la palabra "Date:" y se queda solamente con, en este caso, Sat Jan 15 16:32:28 UTC 2022.
- Luego se busca la línea vacía separa el encabezado del cuerpo. Al encontrarla se cambia la bandera `reading_body` a `True`, por lo que a partir de ese momento, el código sabe que las próximas líneas ya pertenecen al cuerpo Base64.
- Cuando se lee el cuerpo, se toma cada línea del cuerpo y se la agrega a la lista `body_base64`.

3.2. Se valida si se encontró la fecha o no, de igual forma si se encontró o no el cuerpo Base64.

4. Finalmente se devuelve la fecha extraída del encabezado y el cuerpo Base64 completo, uniendo todas sus líneas en una sola cadena.

```
25
26 def separate_challenge(challenge):
27
28     lines = challenge.splitlines()
29
30     date = None
31     body_base64 = []
32     reading_body = False
33
34     for line in lines:
35         if line.startswith("Date:"):
36             date = line.replace("Date:", "").strip()
37
38             if line.strip() == "":
39                 reading_body = True
40                 continue
41
42             if reading_body:
43                 body_base64.append(line.strip())
44
45     if date is None:
46         raise Exception("No se encontró la fecha en el challenge")
47
48     if not body_base64:
49         raise Exception("No se encontró el cuerpo base64 del challenge")
50
51     return date, "".join(body_base64)
52
```

2.2.2. Conversión de la fecha del encabezado a timestamp Unix

El siguiente paso consiste en transformar la fecha que vino en el encabezado del servidor en un número que podamos usar para reconstruir la clave AES. La clave no fue generada directamente con ese texto, sino que fue generada usando el tiempo Unix en microsegundos.

1. Se convierte la fecha del encabezado en un objeto de tipo fecha/hora que Python puede manipular.
2. Se verifica si la fecha tiene zona horaria. Si no la detectó, se la asigna manualmente como UTC. Esto es importante porque el timestamp Unix depende de la zona horaria. Si Python interpreta la fecha como hora local en vez de UTC, el timestamp resultante sería incorrecto y nunca encontraríamos la clave correcta.
3. Se convierte la fecha a timestamp Unix en segundos. Obteniendo un número que representa el segundo exacto informado por el servidor.
4. La clave no fue generada usando segundos, sino microsegundos. Entonces, como un segundo tiene 1 millón microsegundos, se multiplica el timestamp en segundos por 1 millón. Obtenemos como resultado un valor que representa el inicio del segundo informado por el encabezado.
5. Finalmente mostramos por pantalla los resultados obtenidos.

```
95 formatted_date = parsedate_to_datetime(date)
96
97 if formatted_date.tzinfo is None:
98     formatted_date = formatted_date.replace(tzinfo=timezone.utc)
99
100 timestamp_seconds = int(formatted_date.timestamp())
101 timestamp_microseconds_base = timestamp_seconds * 1000000
102
103 print("Fecha del encabezado:", date)
104 print("Timestamp segundos:", timestamp_seconds)
105 print("Timestamp microsegundos inicial:", timestamp_microseconds_base)
106
```

```
Fecha del encabezado: Sat Jan 15 16:32:28 UTC 2022
Timestamp segundos: 1642264348
Timestamp microsegundos: 1642264348000000
```

2.2.3. Decodificación del cuerpo Base64 y separación de sus partes

El objetivo de este paso es tomar el cuerpo Base64 que obtuvimos del servidor y convertirlo nuevamente en bytes reales para poder separar las partes internas del desafío.

1. Se toma el cuerpo en Base64 y lo decodifica. Es decir, pasamos de texto a una secuencia de bytes. Pero esta secuencia de bytes todavía no es el mensaje claro, sino que adentro tiene la estructura completa del desafío:

- [clave simétrica cifrada] [IV] [texto cifrado]

2. Se separan de los bytes obtenidos, los bytes correspondientes a la clave simétrica cifrada. Para esto se toman los primeros 128 bytes, porque RSA es de 1024 bits.
3. Se separan de los bytes obtenidos, los bytes correspondientes al IV, es decir, el vector de inicialización usado por AES-CBC. Para esto se toman los bytes desde la posición 128 a la 143 inclusive, resultando en $144 - 128 = 16$ bytes.
4. Se separan de los bytes obtenidos, los bytes correspondientes al texto cifrado con AES-128-CBC. Para esto se toman todos los bytes desde el byte 144 hasta el final.
5. Finalmente, se muestra el tamaño de bytes de cada parte separada.

```

112 data = base64.b64decode(body_base64)
113
114 encrypted_symmetric_key = data[:128]
115 iv = data[128:144]
116 ciphertext = data[144:]
117
118 print("\nTamaño total del cuerpo Base64:", len(data), "bytes")
119 print("Tamaño de clave simétrica cifrada:", len(encrypted_symmetric_key), "bytes")
120 print("Tamaño de IV:", len(iv), "bytes")
121 print("Tamaño del texto cifrado:", len(ciphertext), "bytes")
122

```

```

Tamaño total del cuerpo Base64: 448 bytes
Tamaño de clave simétrica cifrada: 128 bytes
Tamaño de IV: 16 bytes
Tamaño del texto cifrado: 304 bytes

```

2.2.4. Fuerza bruta sobre los microsegundos posibles, generación de claves y descifrado

Este es el paso central del ataque, ya que falta obtener la clave AES correcta. Como el encabezado nos da la fecha solo hasta los segundos, falta averiguar el microsegundo exacto. Por eso este paso prueba los 1000000 microsegundos posibles dentro de ese segundo.

1. Como primera medida se inicializa el texto claro en una variable plaintext, en None, que sirve para verificar que no se encontró nada.
2. En el ciclo for se prueban todos los microsegundos posibles dentro del segundo indicado por el encabezado. El rango va desde 0 a 999999, porque cubre todos los casos:

```

16:32:28.000001
16:32:28.000002
...
16:32:28.999999

```

2.1. Ahora se construye el timestamp en microsegundos candidato, utilizando el inicio del segundo en microsegundos calculado anteriormente. Entonces, en cada vuelta del ciclo, se le suma un microsegundo distinto.

```
microsecond = 0
timestamp_microseconds = 1642264348000000

microsecond = 1
timestamp_microseconds = 1642264348000001

microsecond = 2
timestamp_microseconds = 1642264348000002

...

microsecond = 892086
timestamp_microseconds = 1642264348892086
```

2.2. Se genera la clave candidata AES a partir del timestamp en microsegundos. Para esto se utiliza un funcion `generate_key()` declarada previamente, la cual hace lo siguiente:

- Toma el timestamp candidato y lo convierte a 8 bytes usando big endian, porque MD5 no se aplica al número como texto, sino a su representación binaria.
- Y a dicho resultado se le aplica MD5, el cual devuelve 16 bytes, es decir, 128 bits. Como AES-128 necesita una clave de 128 bits, esa salida se usa directamente como clave candidata.

```
53
54 def generate_key(timestamp_microseconds):
55
56     seed = timestamp_microseconds.to_bytes(8, byteorder="big")
57     key = hashlib.md5(seed).digest()
58
59     return key
60
```

2.3. Al obtener una clave candidata se realiza el proceso de descifrado utilizando dicha clave, el IV extraído y el texto cifrado. Todos estos datos son pasados por parámetro a otra función declarada previamente `decipher()`, la cual hace lo siguiente:

- Primero crea un descifrador AES en modo CBC, utilizando además la clave candidata y el IV.
- Luego se aplica dicho descifrador al texto cifrado para intentar descifrarlo.
- Pero el resultado todavía puede tener padding PKCS7 al final. Por eso se lo elimina del texto descifrado. Si la clave es incorrecta, lo más probable es que el descifrado genere bytes basura. En ese caso, el padding final no va a tener una estructura PKCS7 válida, y `unpad` lanza un `ValueError`. Por ende, si falla el padding, dicha clave no sirve, y se continúa probando con el siguiente microsegundo.

```

61
62 def decipher(key, iv, ciphertext):
63
64     cipher = AES.new(key, AES.MODE_CBC, iv)
65     decrypted_text = cipher.decrypt(ciphertext)
66
67     text_without_padding = unpad(decrypted_text, AES.block_size)
68
69     return text_without_padding
70

```

3. Se realiza una validación ASCII del texto descifrado obtenido. Si el descifrado no falló, todavía hay que verificar que el resultado sea un mensaje ASCII válido, ya que es lo que indica la consigna. Entonces, se convierten los bytes descifrados a texto ASCII. Si no se puede convertir, significa que probablemente la clave era incorrecta aunque haya pasado el filtro del padding, pero si es posible, entonces probablemente se encontró el mensaje claro.

4. Finalmente, se imprime por pantalla la clave candidata correcta, el microsegundo y el timestamp exacto en microsegundos, sobre el cual se genera la clave candidata correcta, y el mensaje claro obtenido, que luego se lo guarda en una variable message.

```

119 plaintext = None
120
121 for microsecond in range(1000000):
122     timestamp_microseconds = timestamp_microseconds_base + microsecond
123     key = generate_key(timestamp_microseconds)
124
125     try:
126         plaintext_bytes = decipher(key, iv, ciphertext)
127     except ValueError:
128         continue
129
130     try:
131         plaintext = plaintext_bytes.decode("ascii")
132     except UnicodeDecodeError:
133         continue
134
135     print("\nClave encontrada:", key)
136     print("Microsegundo:", microsecond)
137     print("Timestamp en microsegundos:", timestamp_microseconds)
138
139     print("\nMensaje claro:")
140     print(plaintext)
141
142     break
143
144 if plaintext is None:
145     raise Exception("No se encontro la clave correcta")
146
147 message = plaintext
148

```

```

Clave encontrada: b''\xe6\xbb\xe5\xec\xca\x11\xd1f\x9d\xcf,\xf9\xaa\xbd\x1a"
Microsegundo: 892086
Timestamp en microsegundos: 1642264348892086

Mensaje claro:
"The Soviet Union, which has complained recently about alleged anti-Soviet
themes in American advertising, lodged an official protest this week against
the Ford Motor Company's new campaign: 'Hey you stinking fat Russian, get
off my Ford Escort.'"
-- Dennis Miller, Saturday Night Live

```

2.2.5. Envío de la respuesta al servidor

Una vez que se descifró el mensaje, el objetivo ahora es enviarlo al servidor para que valide si el desafío fue resuelto correctamente.

- 1) Se crea la URL que apunta al endpoint donde el servidor espera recibir el mensaje descifrado.
- 2) Luego, se realiza una petición POST en la que se envía el mensaje usando el campo de formulario llamado message.

```
153 response_answer = requests.post(  
154     f"{server}/timerand/{email}/answer",  
155     files={  
156         "message": (None, message)  
157     }  
158 )  
159  
160 print("\nRespuesta del servidor:")  
161 print(f"Status code: {response_answer.status_code}")  
162 print(response_answer.text)  
163
```

- 3) Finalmente, se imprime la respuesta del servidor, lo que permite ver si el envío fue correcto, mostrando el código de estado de la petición http al servidor, por ejemplo, código 200 como en este caso que salió bien, y el contenido de response_answer.text que muestra el mensaje que devuelve el servidor, en este caso ¡Ganaste!, indicando que la actividad fue resuelta correctamente.

```
Respuesta del servidor:  
Status code: 200  
¡Ganaste!
```